



Bitcoin Testnet4

交易与区块逐字节解析实验报告

刘鹤扬 辛汶原 王志歌

2026 年 7 月 20 日

目录

1	小组分工	3
2	摘要	4
3	实验信息	4
4	实验目的	4
5	实验原理	5
5.1	UTXO 与交易	5
5.2	Locking Script 与 Unlocking Data	5
5.3	txid、wtxid 与交易重量	5
5.4	区块、Merkle root 与 PoW	6
6	实验环境初始化	6
7	获取 Testnet4 UTXO	7
7.1	生成 faucet 地址	7
7.2	Faucet 发币	8
7.3	确认 UTXO	8
8	构造并发送自转账	9
9	交易逐字节解析	10
9.1	交易总体信息	10
9.2	Input 与 witness	11
9.3	Outputs、找零和手续费	12
9.4	Byte/bit 布局	12
10	完整区块解析与验证	13
10.1	区块基本信息	14
10.2	Merkle root 验证	14
10.3	PoW 验证	15
11	本地自动化测试	15

12 实验结果与讨论	16
13 实验结论	16
参考资料	17

1 小组分工

姓名	学号	分工
刘鹤扬	202300180102	编写测试脚本，testnet 测试，编写报告
王志歌	202300460119	编写测试脚本，本地测试
辛汶原	202300460115	独立完成任务 2，未参与当前任务

2 摘要

本实验在 Bitcoin testnet4 上运行 Bitcoin Core 完整验证节点，通过 JSON-RPC 创建钱包、从 faucet 获取测试币，并完成一笔同钱包 P2WPKH 地址间的真实 SegWit 交易。实验程序对该交易的 222 个 byte 进行了字段级和 bit-range 级映射，计算了 txid、wtxid、weight 和 vsize，并反汇编了 locking/unlocking data。目标交易随后在高度 144917 的区块中得到确认；程序从完整 raw block 解析 35 笔交易，独立重算 Merkle root，并将区块头的 nBits 展开为 PoW target。交易、Merkle root 与 PoW 校验均成功，9 项自动化测试全部通过。

3 实验信息

项目	内容
实验名称	Bitcoin Testnet4 交易发送、序列化解析与区块验证
实验日期	2026-07-20
实验网络	Bitcoin testnet4
实验环境	Windows PowerShell、Bitcoin Core、Python 3.14
钱包名称	task1
节点模式	完整验证、pruned 存储

4 实验目的

本实验将 Bitcoin 的 UTXO、交易、Script、区块和 PoW 连接为一个完整流程，主要目标如下：

1. 运行 Bitcoin Core testnet4 节点，并通过 JSON-RPC 操作节点和钱包。
2. 从 faucet 获取 testnet4 测试币，形成由本钱包控制的 UTXO。
3. 使用该 UTXO 构造并广播一笔同钱包地址间的真实测试网交易。
4. 解析交易的 version、input、output、witness 和 locktime，并将每个 byte 映射到字段及 bit 范围。
5. 解析包含目标交易的完整区块，重新计算 block hash、Merkle root 和 PoW target。
6. 通过自动化测试验证 legacy、SegWit、Script、CompactSize 和异常输入处理。

5 实验原理

5.1 UTXO 与交易

Bitcoin 不直接维护账户余额。一笔交易的 input 通过 `previous txid + vout` 引用旧交易的某个未花费输出 (UTXO)，output 则创建新的金额和锁定条件。输入金额之和减去输出金额之和构成矿工费。

普通交易的基本序列化顺序为：

```
version
[marker + flag, for SegWit]
input count + inputs
output count + outputs
[witness stacks, for SegWit]
locktime
```

多字节整数通常使用小端序，input/output 数量和 Script 长度使用 CompactSize 编码。

5.2 Locking Script 与 Unlocking Data

本实验使用原生 P2WPKH。其 locking Script 形式为：

```
OP_0 <20-byte public-key hash>
```

对应原始字节为 `00 14 <20 bytes>`。花费 P2WPKH UTXO 时，legacy `scriptSig` 为空，签名和压缩公钥放在 witness stack 中。节点检查公钥的 HASH160 是否等于 witness program，并验证签名。

5.3 txid、wtxid 与交易重量

SegWit 交易具有两个哈希标识：

- **txid**: 对不含 marker、flag 和 witness 的基础序列化做双 SHA-256。
- **wtxid**: 对包含 witness 的完整序列化做双 SHA-256。

交易重量按下式计算：

$$\text{weight} = 3 \times \text{base_size} + \text{total_size}, \quad (1)$$

$$\text{vsize} = \left\lceil \frac{\text{weight}}{4} \right\rceil. \quad (2)$$

5.4 区块、Merkle root 与 PoW

完整区块由 80-byte header、CompactSize 交易数量和所有序列化交易组成。区块头包含 version、previous block hash、Merkle root、timestamp、nBits 和 nonce。

本实验执行三项验证：

1. 对 80-byte header 做双 SHA-256，得到显示顺序的 block hash。
2. 使用区块中所有交易的 txid 重建 Merkle tree，并与 header 中的 Merkle root 比较。
3. 将 nBits 展开为 256-bit target，验证 $\text{block hash} \leq \text{target}$ 。

6 实验环境初始化

首先以 testnet4、RPC server 和 pruning 模式启动 Bitcoin Core：

```
$CoreArgs = "-testnet4 -server -prune=2000 -datadir=\"$DataDir\""
Start-Process -FilePath $Bitcoind -ArgumentList $CoreArgs -WindowStyle Hidden
& $BitcoinCli -testnet4 "-datadir=$DataDir" -rpcwait getrpcinfo
```

getrpcinfo 成功返回，说明外部 RPC 已启动。随后查询链状态，结果如表 1 所示。

表 1 Bitcoin Core testnet4 同步状态

字段	实际值
chain	testnet4
blocks	144912
headers	144912
verificationprogress	1
initialblockdownload	False
pruned	True

这表明节点已同步到当时链尖，并且正在使用 pruned 存储。之后创建 **task1** 钱包，通过 **listwallets** 确认钱包已加载。

```
PS C:\Users\Dolan duck\Desktop\。 \学习\2026-2027\暑期课程\Day1> $CoreArgs = "-testnet4 -server -prune=2000 -datadir=~\"$DataDir\"""
PS C:\Users\Dolan duck\Desktop\。 \学习\2026-2027\暑期课程\Day1> Start-Process -FilePath $BitcoinCmd -ArgumentList $CoreArgs -WindowStyle Hidden
PS C:\Users\Dolan duck\Desktop\。 \学习\2026-2027\暑期课程\Day1> & $BitcoinCli -testnet4 "-datadir=$DataDir" -rpcwait getrpcinfo
{
  "active_commands": [
    {
      "method": "getrpcinfo",
      "duration": 15
    }
  ],
  "logpath": "E:\\Bitcoin\\Data\\testnet4\\debug.log"
}
PS C:\Users\Dolan duck\Desktop\。 \学习\2026-2027\暑期课程\Day1> $Chain = & $BitcoinCli -testnet4 "-datadir=$DataDir" getblockchaininfo |
>> ConvertFrom-Json
PS C:\Users\Dolan duck\Desktop\。 \学习\2026-2027\暑期课程\Day1> $Chain | Select-Object chain, blocks, headers, verificationprogress, initialblockdownload, pruned

chain           : testnet4
blocks          : 144912
headers         : 144912
verificationprogress : 1
initialblockdownload : False
pruned          : True

PS C:\Users\Dolan duck\Desktop\。 \学习\2026-2027\暑期课程\Day1> & $BitcoinCli -testnet4 "-datadir=$DataDir" createwallet "task1"
{
  "name": "task1"
}
PS C:\Users\Dolan duck\Desktop\。 \学习\2026-2027\暑期课程\Day1> & $BitcoinCli -testnet4 "-datadir=$DataDir" listwallets
[
  "task1"
]
```

图 1 Bitcoin Core RPC、testnet4 同步状态和 task1 钱包创建

7 获取 Testnet4 UTXO

7.1 生成 faucet 地址

使用 **task1** 钱包生成 Bech32 地址：

```
$FaucetAddress = (
    & $BitcoinCli -testnet4 "-datadir=$DataDir" -rpcwallet=task1 `
    getnewaddress "faucet-in" "bech32"
).Trim()
```

生成的地址为：

```
tb1qjz8mevqtj526tq9reau6r4yc9nw92sztkhezy9
```

```
PS C:\Users\Dolan duck\Desktop\。 \学习\2026-2027\暑期课程\Day1> $FaucetAddress = (
>> & $BitcoinCli -testnet4 "-datadir=$DataDir" -rpcwallet=task1 `
>> getnewaddress "faucet-in" "bech32"
>> ).Trim()
PS C:\Users\Dolan duck\Desktop\。 \学习\2026-2027\暑期课程\Day1> $FaucetAddress
tb1qjz8mevqtj526tq9reau6r4yc9nw92sztkhezy9
```

图 2 通过 wallet RPC 生成标签为 faucet-in 的 P2WPKH 收款地址

7.2 Faucet 发币

将该地址提交至 testnet4.info faucet，页面显示发送成功，faucet 交易 txid 为：

```
97a5ecff674dd945e663d714296b1ac00fab99dc19f4d2977629337a8bcb4258
```

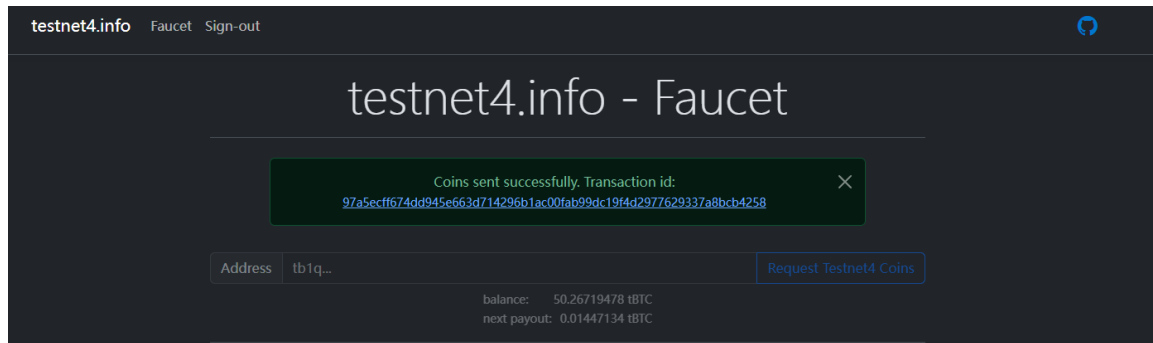


图 3 Faucet 成功广播 testnet4 交易并返回 txid

7.3 确认 UTXO

等待交易进入区块后执行：

```
& $BitcoinCli -testnet4 "-datadir=$DataDir" -rpcwallet=task1 listunspent 1
```

等待约 50 分钟后，钱包返回一个有 1 次确认的 UTXO，具体信息如表 2 所示。

表 2 Faucet UTXO 信息

字段	实际值
txid	97a5ecff674dd945e663d714296b1ac00fab99dc19f4d2977629337a8bcb4258
vout	1
amount	0.01448372 tBTC (1,448,372 sat)
confirmations	1
spendable / solvable / safe	True / True / True
output type	P2WPKH

```
PS C:\Users\Dolan duck\Desktop\o\学习\2026-2027\暑期课程\Day1> & $BitcoinCli -testnet4 "-datadir=$DataDir" -rpcwallet=task1 listunspent 1
[
  {
    "txid": "97a5ecff674dd945e663d714296b1ac00fab99dc19f4d2977629337a8bcb4258",
    "vout": 1,
    "address": "tb1qjz8mevqtj526tq9reau6n4yc9nw92sztkhezy9",
    "label": "faucet-in",
    "scriptPubKey": "0014908fbc0b0b9515a580a3cf79a1d4982cdc55404b",
    "amount": 0.01448372,
    "confirmations": 1,
    "spendable": true,
    "solvable": true,
    "desc": "wpkh([3b4d3470/84h/1h/0h/0]02d7caa4c32bcf0f99fd59b79000a4bbcb687366819dfcd07352146bae54e97)#rg0yq9zs",
    "parent_descs": [
      "wpkh([3b4d3470/84h/1h/0h]tpubDCRFByEF1scqN6JB5vuCuK583Znuh1W2GFMpb1af6QfjEd1EcPfmFAD3LyOQdaAkBG1Bnh1h2UyYsPvrXBD6H#V2wo6GbP3ja5w6M1AH7tx/0/*)#ewk051ef"
    ],
    "safe": true
  }
]
```

图 4 Faucet 输出已经确认，并由 task1 钱包控制和安全花费

8 构造并发送自转账

在同一钱包内创建标签为 **task1-self-receive** 的第二个 Bech32 地址作为交易接收地址：

```
tb1qxsapsm219u74dvzlqssaatau585ecc0wuj4xv1
```

发送前再次调用 **getblockchaininfo**，确认当前链为 **testnet4**。随后调用：

```
$Txid = (
  & $BitcoinCli -testnet4 "-datadir=$DataDir" -rpcwallet=task1 `
    sendtoaddress $ReceiveAddress 0.00010000
).Trim()
```

广播得到的目标交易 txid 为：

```
5ce1ab1b4e49ed35b1ebad7fc1eca08320aa2cbe0a7dfd222930bde170c65806
```

```
PS C:\Users\Dolan duck\Desktop\o\学习\2026-2027\暑期课程\Day1> $ReceiveAddress = (
>> & $BitcoinCli -testnet4 "-datadir=$DataDir" -rpcwallet=task1 `
>> getnewaddress "task1-self-receive" "bech32"
>> ).Trim()
PS C:\Users\Dolan duck\Desktop\o\学习\2026-2027\暑期课程\Day1> $ReceiveAddress
tb1qxsapsm219u74dvzlqssaatau585ecc0wuj4xv1
PS C:\Users\Dolan duck\Desktop\o\学习\2026-2027\暑期课程\Day1> (& $BitcoinCli -testnet4 "-datadir=$DataDir" getblockchaininfo |
>> ConvertFrom-Json).chain
testnet4
PS C:\Users\Dolan duck\Desktop\o\学习\2026-2027\暑期课程\Day1> $Txid = (
>> & $BitcoinCli -testnet4 "-datadir=$DataDir" -rpcwallet=task1 `
>> sendtoaddress $ReceiveAddress 0.00010000
>> ).Trim()
PS C:\Users\Dolan duck\Desktop\o\学习\2026-2027\暑期课程\Day1>
PS C:\Users\Dolan duck\Desktop\o\学习\2026-2027\暑期课程\Day1> $Txid
5ce1ab1b4e49ed35b1ebad7fc1eca08320aa2cbe0a7dfd222930bde170c65806
```

图 5 向同钱包新地址发送 0.00010000 tBTC，Bitcoin Core 返回真实 testnet4 txid

9 交易逐字节解析

通过 `gettransaction` 获取 raw transaction，再运行本项目解析器：

```
$WalletTx = & $BitcoinCli -testnet4 "-datadir=$DataDir" -rpcwallet=task1 `
    gettransaction $Txid | ConvertFrom-Json
```

```
$RawTx = $WalletTx.hex
```

```
python -m bitcoin_parser tx --hex $RawTx --bytes |
    Tee-Object "tx-$Txid.txt"
```

```
PS C:\Users\Dolan duck\Desktop> .\学习\2026-2027\暑期课程\Day1> $WalletTx = & $BitcoinCli -testnet4 "-datadir=$DataDir" -rpcwallet=task1 `
>> gettransaction $Txid | ConvertFrom-Json
PS C:\Users\Dolan duck\Desktop> .\学习\2026-2027\暑期课程\Day1> $RawTx = $WalletTx.hex
PS C:\Users\Dolan duck\Desktop> .\学习\2026-2027\暑期课程\Day1>
PS C:\Users\Dolan duck\Desktop> .\学习\2026-2027\暑期课程\Day1> $RawTx
0200000000001015842cb8b7a33297697d2f419dc99ab0fc01a6b2914d763e645d94d67ffeca5970100000000fdffffff021027000000000000160014343a186d5f2f3d56b05ff
4a2c90396b0f6708776d840247304402201effbc13fec66b2c1f09833a881b27320f99c041e8ff09f1137e90b61979a890022070b8906730d6877e5ce19785c3fc2d62caa32ff
4bbbc687366819dfcd07352146beae54e9712360200
PS C:\Users\Dolan duck\Desktop> .\学习\2026-2027\暑期课程\Day1> Set-Location $Project
PS C:\Users\Dolan duck\Desktop> .\学习\2026-2027\暑期课程\Day1\task1> python -m bitcoin_parser tx --hex $RawTx --bytes |
>> Tee-Object "tx-$Txid.txt"
TRANSACTION
txid:      5ce1ab1b4e49ed35b1ebad7fc1eca08320aa2cbe0a7dfd222930bde170c65806
wtxid:     dd14621c0d835fcb83bc4c515370797a78dbb04d91700ce5f42c510c5cf3dcd1
format:    version 2, SegWit
size:      222 bytes, 561 WU, 141 vbytes
coinbase:  False
locktime:  144914
inputs:    1
```

图 6 获取 raw transaction 并运行逐字节解析器

完整输出保存在：

`results/tx-5ce1a...0c65806.txt`

9.1 交易总体信息

字段	解析结果
txid	5ce1ab1b4e49ed35b1ebad7fc1eca08320aa2cbe0a7dfd222930bde170c65806
wtxid	dd14621c0d835fcb83bc4c515370797a78dbb04d91700ce5f42c510c5cf3dcd1
version	2
格式	SegWit, marker/flag = 00 01
input / output	1 / 2

字段	解析结果
total size	222 bytes
base size	113 bytes
witness size	109 bytes
weight	561 WU
vsize	141 vbytes
locktime	144914
coinbase	False

txid 与 wtxid 不同，验证了 witness 不参与传统 txid、但参与 wtxid 的计算。

9.2 Input 与 witness

唯一 input 引用 faucet 交易的第 1 号输出：

```
97a5ecff674dd945e663d714296b1ac00fab99dc19f4d2977629337a8bcb4258:1
```

该 UTXO 金额为 1,448,372 sat。解析结果显示：

- scriptSig 为空，符合原生 SegWit P2WPKH 特征。
- sequence 为 0xffffffff。
- witness item count 为 2。
- witness 第 1 项是 71-byte DER ECDSA signature，末尾 01 为 sighash type。
- witness 第 2 项是 33-byte compressed public key。

9.3 Outputs、找零和手续费

表 4 目标交易输出

vout	金额	类型	地址	作用
0	10,000 sat	P2WPKH	tb1qxsapsm2l9u74dvz1qssaatau585ecc0wuj4xv1	自转账目标输出
1	1,438,230 sat	P2WPKH	tb1ql5yld7rvhtuy3sj29jgrj6c0vuy8wmvy3r27yr	钱包找零输出

手续费由输入减去输出得到:

$$\text{fee} = 1,448,372 - 10,000 - 1,438,230 = 142 \text{ sat}, \quad (3)$$

$$\text{fee rate} = \frac{142}{141} \approx 1.007 \text{ sat/vB}. \quad (4)$$

两个 output 的 locking Script 均为 22-byte P2WPKH:

```
OP_0 PUSHBYTES_20[public-key-hash]
```

9.4 Byte/bit 布局

解析器为 222 个 byte 全部生成了字段映射, 主要区间如表 5 所示。

表 5 目标交易 Byte/bit 布局

Byte 范围	Bit 范围	内容
0..3	0..31	version
4..5	32..47	SegWit marker / flag
6..47	48..383	input count 与唯一 input
48	384..391	output count
49..79	392..639	目标 output
80..110	640..887	找零 output
111..217	888..1743	witness item count、签名和公钥
218..221	1744..1775	locktime

这使每个 byte 都可以追溯到具体协议字段，而不仅是输出一段 raw hex。

10 完整区块解析与验证

为了避免手动反复查询交易是否被确认，使用轮询脚本捕获确认状态：

```
do {
    Start-Sleep -Seconds 30

    $WalletTx = & $BitcoinCli -testnet4 "-datadir=$DataDir" -rpcwallet=task1 `
        gettransaction $Txid | ConvertFrom-Json

    Write-Host "Confirmations:" $WalletTx.confirmations
} while ($WalletTx.confirmations -lt 1)
```

等待约 30 分钟后得到确认。目标交易所在区块的 hash 为:

```
00000000000000000010fd4dcdf15b1087d663e3030f6917d3becb43ab22465b25
```

通过 Bitcoin Core RPC 获取完整 raw block，并分别输出逐字节报告和 JSON：

```
python -m bitcoin_parser fetch-block $BlockHash `
--rpc-url "http://127.0.0.1:48332" `
--rpc-cookie $Cookie `
--bytes |
Tee-Object "block-$BlockHash.txt"

python -m bitcoin_parser fetch-block $BlockHash `
--rpc-url "http://127.0.0.1:48332" `
--rpc-cookie $Cookie `
--json |
Tee-Object "block-$BlockHash.json"
```

完整结果保存在以下文件中:

- results/block-0000...17d3becb43ab22465b25.txt
- results/block-0000...17d3becb43ab22465b25.json

目标交易是该区块中的第 14 笔交易（从 0 开始的索引为 13）。

10.1 区块基本信息

字段	解析结果
height	144917
block hash	000000000000000010fd4dcdf15b1087d663e3030f6917d3becb43ab22465b25
version	593641472
previous block	000000000e0669759581d1d3fe716fc329e6a7924d8b50d7d4a46b30210ce37
timestamp	1784554522 (2026-07-20T13:35:22Z)
bits	0x190228f4
nonce	3497288800
transaction count	35
raw size	82,923 bytes
stripped size	4,994 bytes
weight	97,905 WU
vsize	24,477 vbytes
first transaction coinbase	True

10.2 Merkle root 验证

区块头记录值与本地重算值完全相同：

```
header Merkle root:
7f2d432e68ff3cccfdc1a50499f39990d2b5635d0683749ef99e3f960b4596d4

calculated Merkle root:
7f2d432e68ff3cccfdc1a50499f39990d2b5635d0683749ef99e3f960b4596d4

merkle_root_valid = True
```

这说明区块 JSON 中解析得到的 35 笔交易与 header 的 Merkle commitment 一致，目标交易也确实被承诺在该区块中。

10.3 PoW 验证

解析 nBits = 0x190228f4 得到：

```
target =
0000000000000000228f40000000000000000000000000000000000000000000

proof value / block hash =
0000000000000000010fd4dcdf15b1087d663e3030f6917d3becb43ab22465b25

proof_of_work_valid = True
```

因为 proof value 小于 target，PoW 验证通过。解析器计算的相对 difficulty 约为 1.9884×10^9 。

11 本地自动化测试

运行本地项目测试：

```
python -m unittest discover -s tests -v
```

```
PS C:\Users\Dolan duck\Desktop\。学习\2026-2027\暑期课程\Day1\task1> Set-Location $Project
PS C:\Users\Dolan duck\Desktop\。学习\2026-2027\暑期课程\Day1\task1> python -m unittest discover -s tests -v
test_rejects_trailing_block_data (test_parser.BlockParserTests.test_rejects_trailing_block_data) ... ok
test_testnet3_genesis_block (test_parser.BlockParserTests.test_testnet3_genesis_block) ... ok
test_classifies_taproot (test_parser.ScriptTests.test_classifies_taproot) ... ok
test_disassembles_push_and_checksigs (test_parser.ScriptTests.test_disassembles_push_and_checksigs) ... ok
test_matches_bip173_testnet_address_vector (test_parser.ScriptTests.test_matches_bip173_testnet_address_vector) ... ok
test_legacy_transaction (test_parser.TransactionParserTests.test_legacy_transaction) ... ok
test_rejects_noncanonical_compact_size (test_parser.TransactionParserTests.test_rejects_noncanonical_compact_size) ... ok
test_rejects_truncated_data (test_parser.TransactionParserTests.test_rejects_truncated_data) ... ok
test_segwit_transaction_has_distinct_ids_and_weight (test_parser.TransactionParserTests.test_segwit_transaction_has_distinct_ids_and_weight) ... ok

-----
Ran 9 tests in 0.002s

OK
```

图 7 解析器的 9 项自动化测试全部通过

测试覆盖：

- testnet3 创世块 hash、Merkle root 与 PoW；
- legacy 与 SegWit 交易；
- txid、wtxid、weight 和 vsize；
- P2PKH、P2TR、BIP173 地址和 Script push；
- 非规范 CompactSize、截断输入和尾随区块数据。

最终结果为：


```
Ran 9 tests in 0.002s  
OK
```

12 实验结果与讨论

本实验完成了从 UTXO 获取到区块确认的闭环：

同步 testnet4 节点 → 创建 wallet 和 P2WPKH 地址 → faucet 产生已确认 UTXO → wallet 选择 UTXO、创建找零并签名 → 广播 SegWit 交易 → 逐 byte 解析 raw transaction → 交易进入高度 144917 的区块 → 解析完整区块并验证 Merkle root 与 PoW。

交易 **scriptSig** 为空而 witness 含签名和公钥，两个 output 均为 **OP_0 + 20-byte hash**，体现了原生 P2WPKH 的 SegWit 数据布局。**txid** 与 **wtxid** 不同，也直接反映 witness 数据与传统交易标识的分离。

区块解析程序从完整 raw block 解析 35 笔交易，使用解析出的 txid 重算 Merkle root，并从 header 的 **nBits** 独立展开 target 后验证 PoW。

13 实验结论

本实验在 Bitcoin testnet4 上成功获得测试币、创建并广播真实 SegWit 交易，随后完成了交易 222 个 byte 的逐字段/逐 bit-range 映射。目标交易在高度 144917 的区块中得到确认，区块 Merkle root 和 PoW 均通过本地重新计算验证。自动化测试 9 项全部通过，实验目标完成。

参考资料

- [1] Bitcoin Developer Reference: Transactions.
<https://developer.bitcoin.org/reference/transactions.html>
- [2] Bitcoin Developer Reference: Block Chain.
https://developer.bitcoin.org/reference/block_chain.html
- [3] Bitcoin Core RPC Reference.
<https://bitcoincore.org/en/doc/30.0.0/rpc/>
- [4] BIP 141: Segregated Witness.
<https://github.com/bitcoin/bips/blob/master/bip-0141.mediawiki>